

程序设计实践

The Practice of Programming

设计与实现

设计实现一个脚本解释器

脚本（Script）

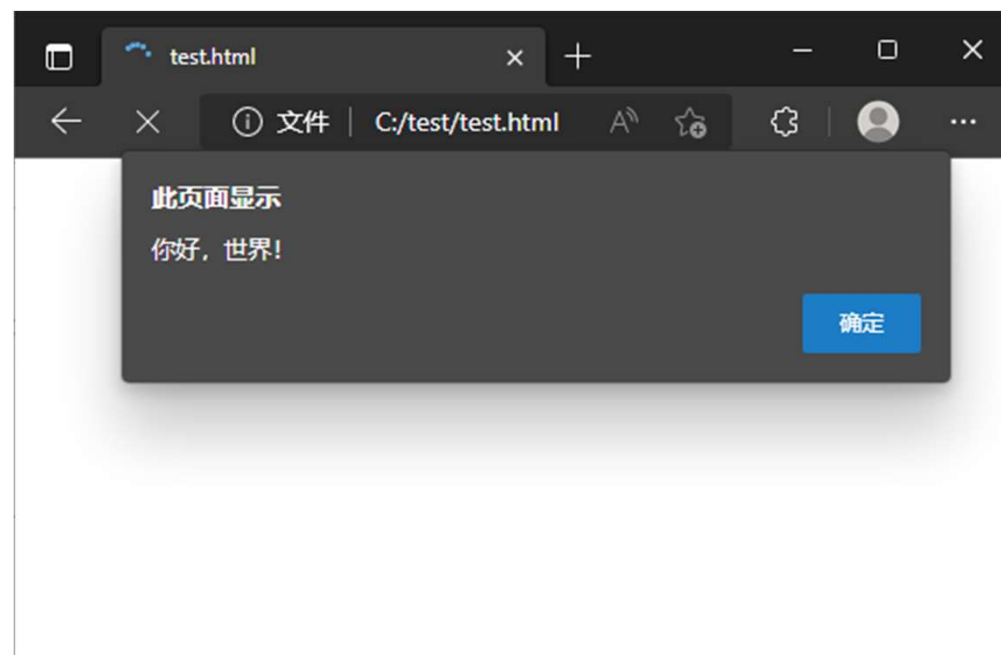
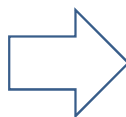
为了特定目的，使用特定语法编写的一个文本文件，描述了一个流程，可以被一个解释器解释执行，以完成特定的功能。

例如：

- Bourne Again Shell脚本，使用/bin/bash解释执行
- C Shell脚本，使用/bin/csh解释执行
- JavaScript脚本，使用浏览器解释执行
- Python脚本，使用python解释执行
- perl脚本，使用perl解释执行
- ...

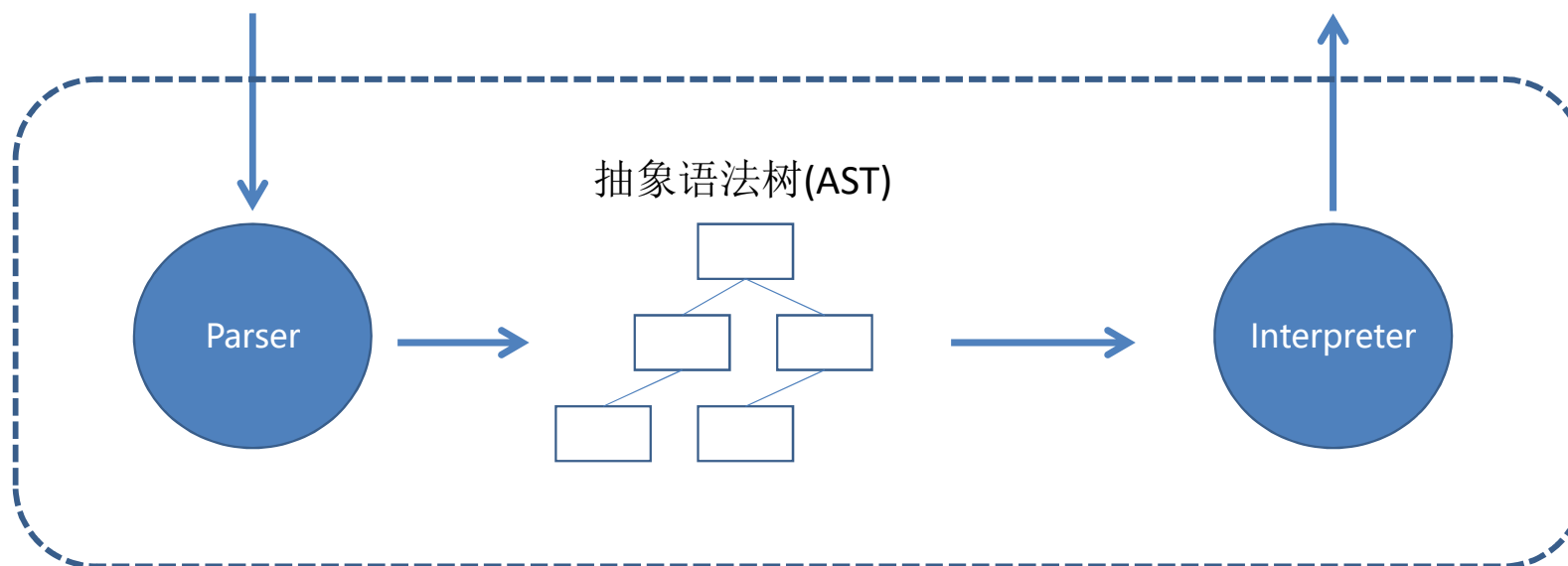
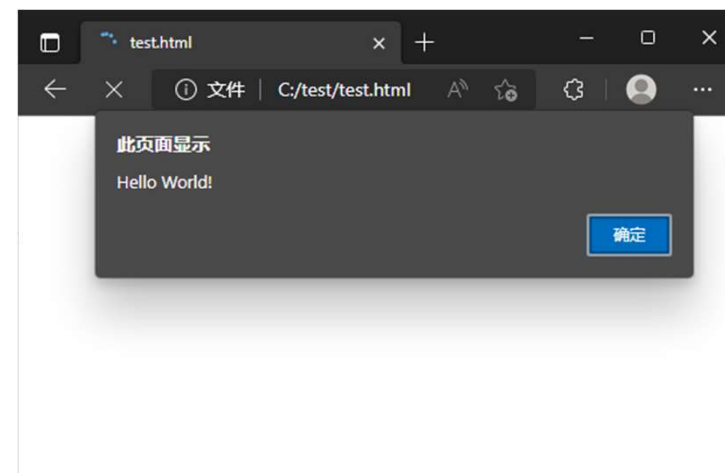
一个简单的脚本示例

```
<script>  
  alert("Hello World!");  
  alert("你好, 世界! ");  
</script>
```



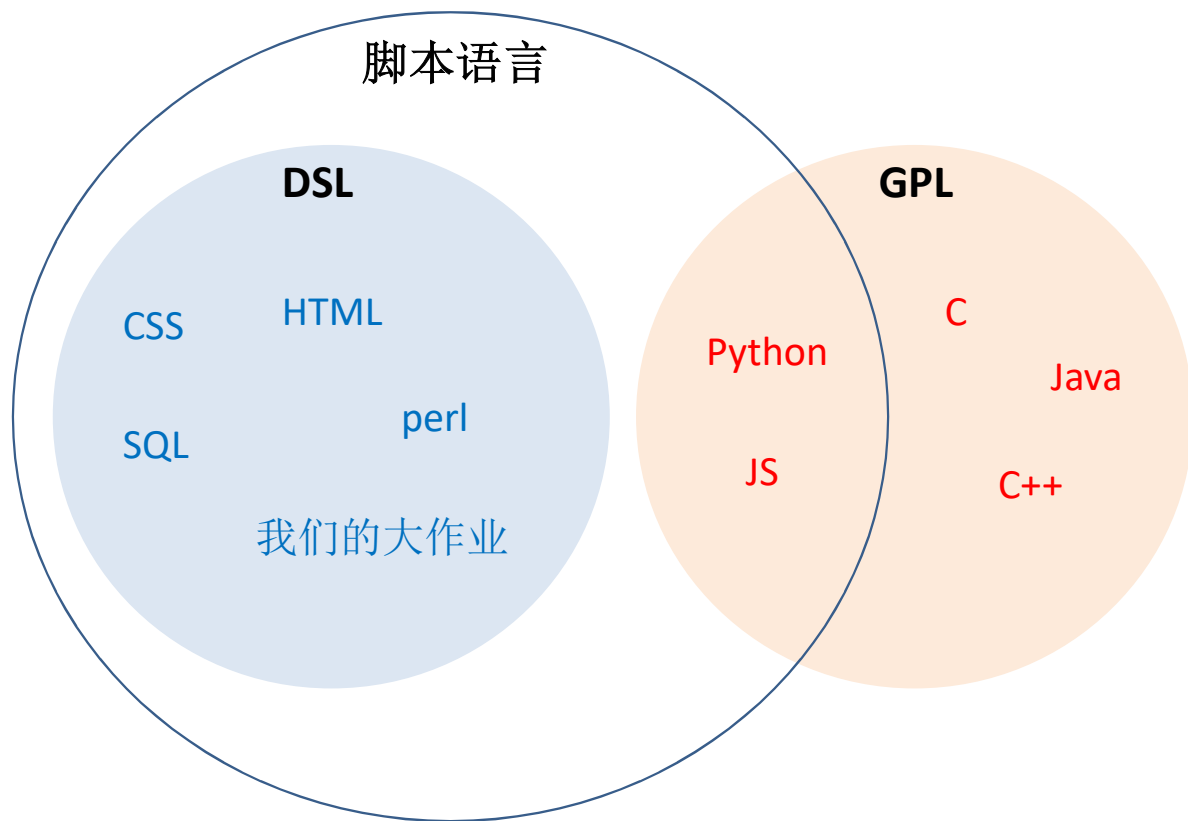
一个简单的脚本示例

```
<script>
  alert("Hello World!");
  alert("你好, 世界! ");
</script>
```

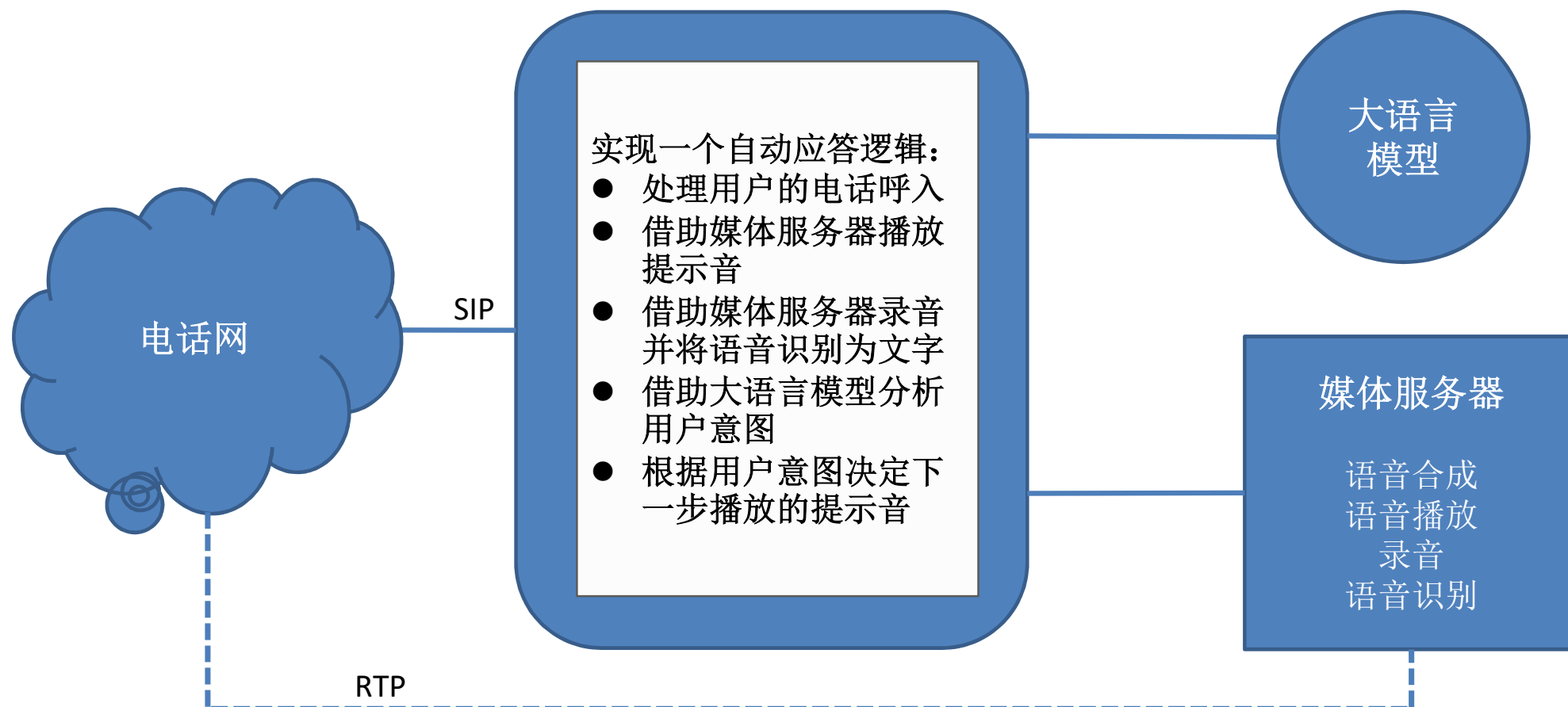


DSL和GPL

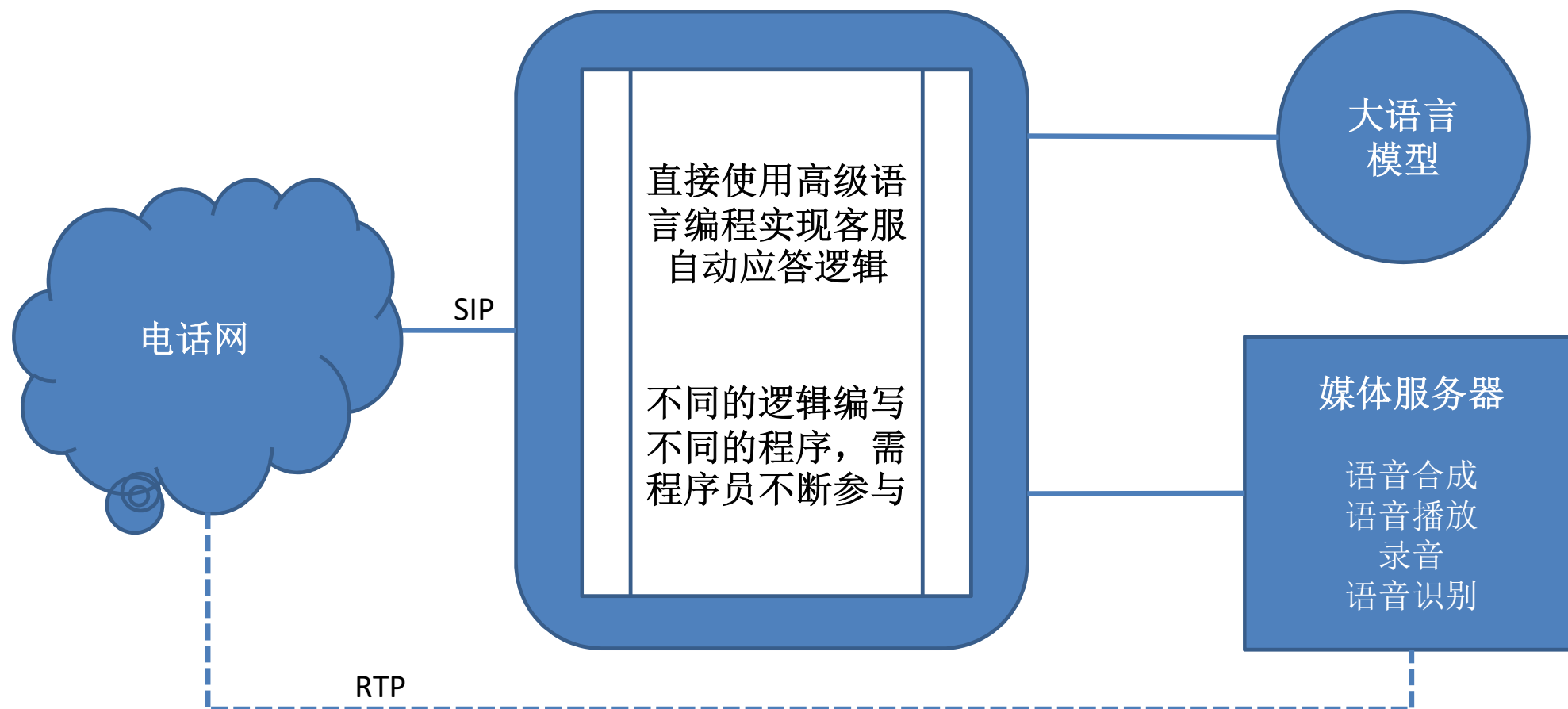
- DSL,
Domain Specific Language,
领域特定语言
- GPL,
General Purpose Language,
通用编程语言



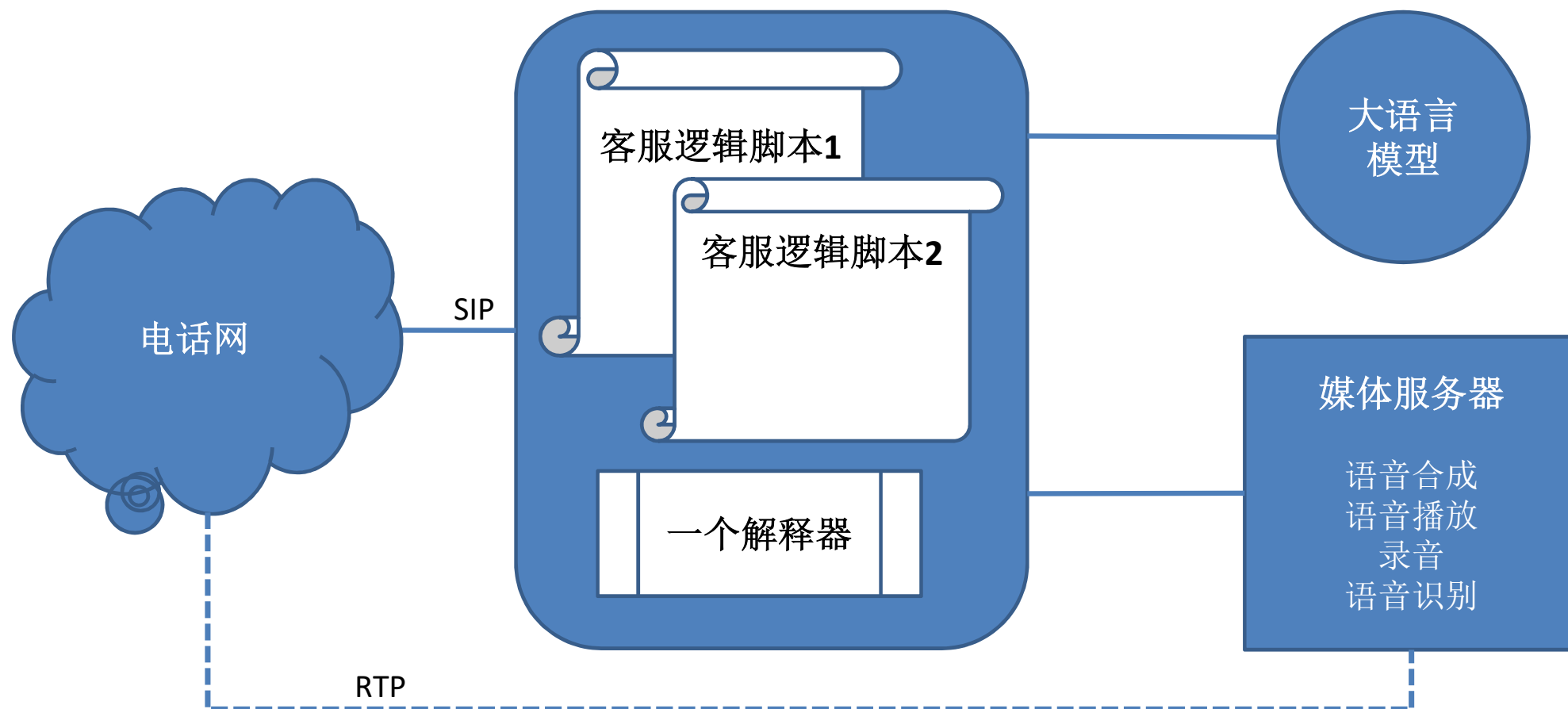
需求 - 实现一个语音客服机器人



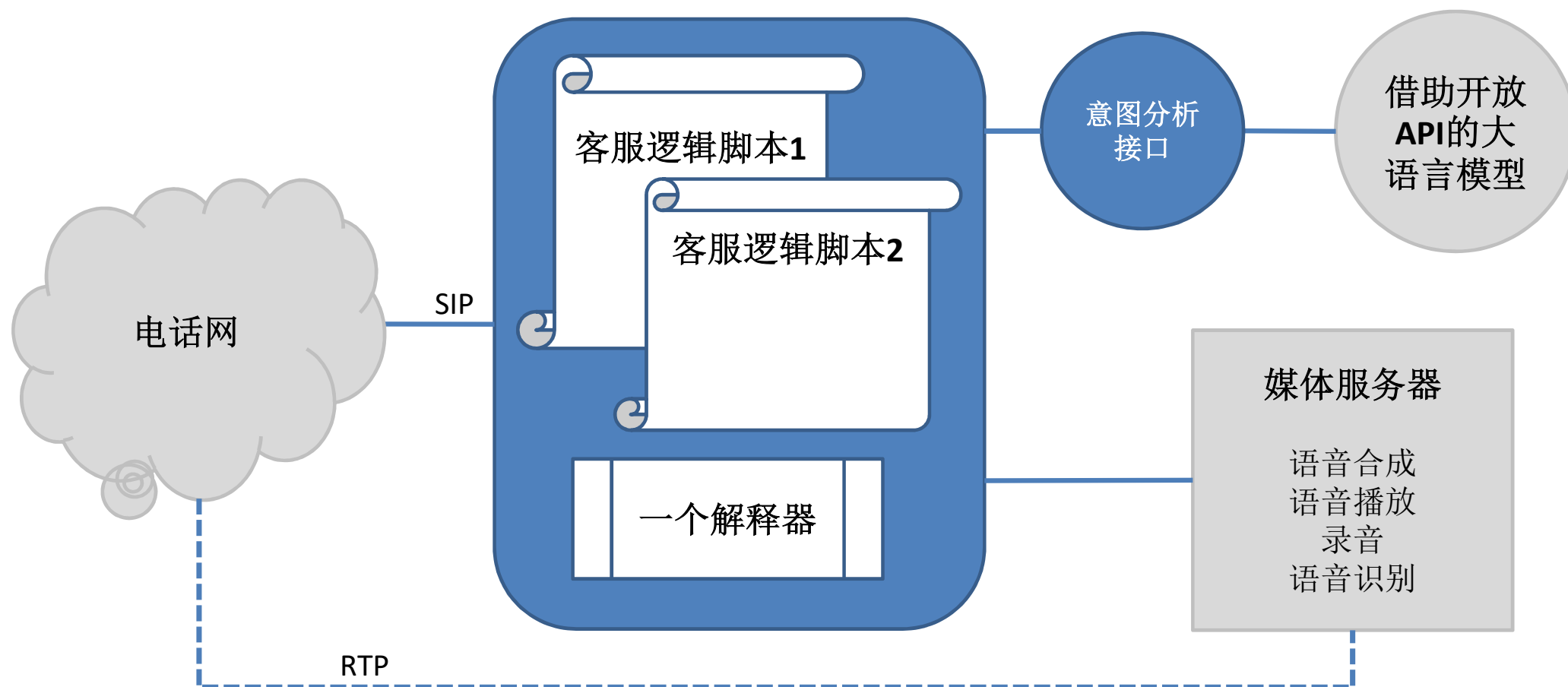
方案1



方案2



我们要做的部分

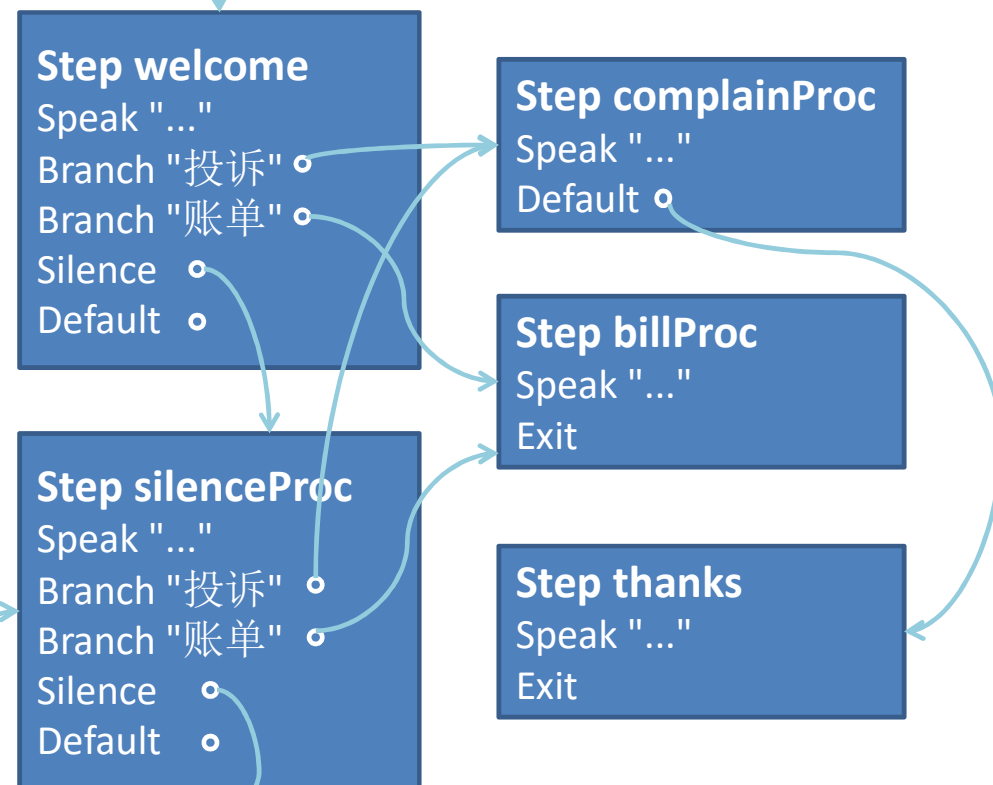


客服逻辑脚本示例

```

Step welcome
  Speak $name + "您好, 请问有什么可以帮您?"
  Listen 5, 20
  Branch "投诉", complainProc
  Branch "账单", billProc
  Silence silenceProc
  Default defaultProc
Step complainProc
  Speak "您的意见是我们改进工作的动力, 请问您还有什么补充?"
  Listen 5, 50
  Default thanks
Step thanks
  Speak "感谢您的来电, 再见"
  Exit
Step billProc
  Speak "您的本月账单是" + $amount + "元, 感谢您的来电, 再见"
  Exit
Step silenceProc
  Speak "听不清, 请您大声一点可以吗"
  Listen 5, 20
  Branch "投诉", complainProc
  Branch "账单", billProc
  Silence silenceProc
  Default defaultProc
Step defaultProc
  ....
  
```

入口



脚本的语义动作

Step: 完整表示一个步骤的所有行为

Speak: 计算表达式合成一段文字。调用媒体服务器进行语音合成并播放

Listen: 调用媒体服务器对客户说的话录音，并进行语音识别。语音识别的结果调用“自然语言分析服务”分析客户的意愿

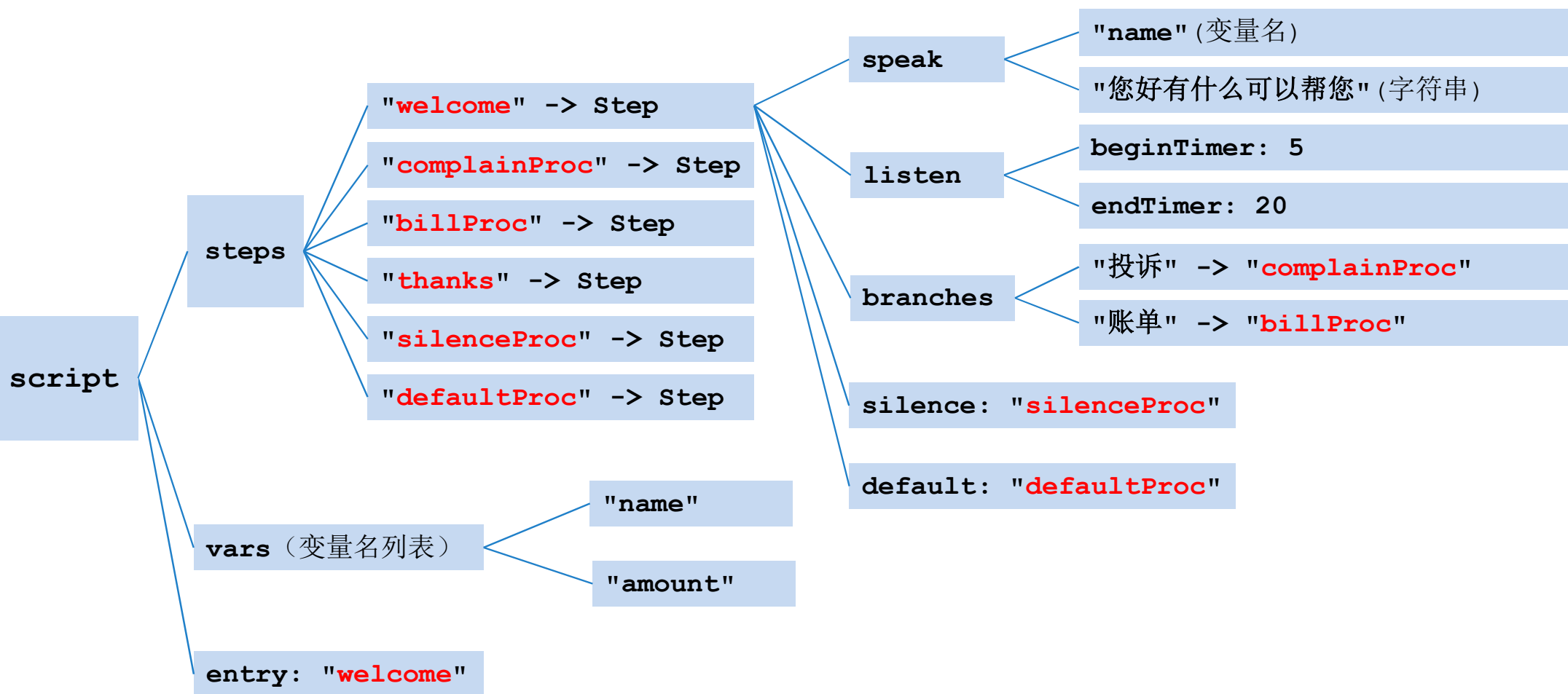
Branch: 对客户的意愿进行分支处理，不同的意愿，跳转到不同的**Step**

Silence: 如果用户不说话，应该跳转到哪个**Step**

Default: 如果客户意愿没有相应匹配，应该跳转到哪个**Step**

Exit: 结束对话

将脚本的语法元素抽象为树形结构



数据结构

数组（使用下标访问，最简单，最快，最常用的“算法”）

- 要查询的数据的索引都是不太大的整数。
- 这些整数稀疏程度不高。

线性表 - 顺序查找（简单，低效的查找算法）

- 要查询的数据特别少，例如少于10条。
- 被查找数据的排列顺序不能自由设定，例如字符串中查找。
- 查询条件太复杂，以至于匹配条件的开销大于查找。
- 更新频繁，却很少查找。

有序线性表（可以二分查找，高效，但是对应用条件要求比较苛刻）

- 数据已经有序
- 数据基本有序，查找前可以很快调整成有序
- 数据无序，但是排序后更新次数远小于查找次数。

HASH表（计算机科学领域伟大的发明之一，应用最广泛的一个数据结构）

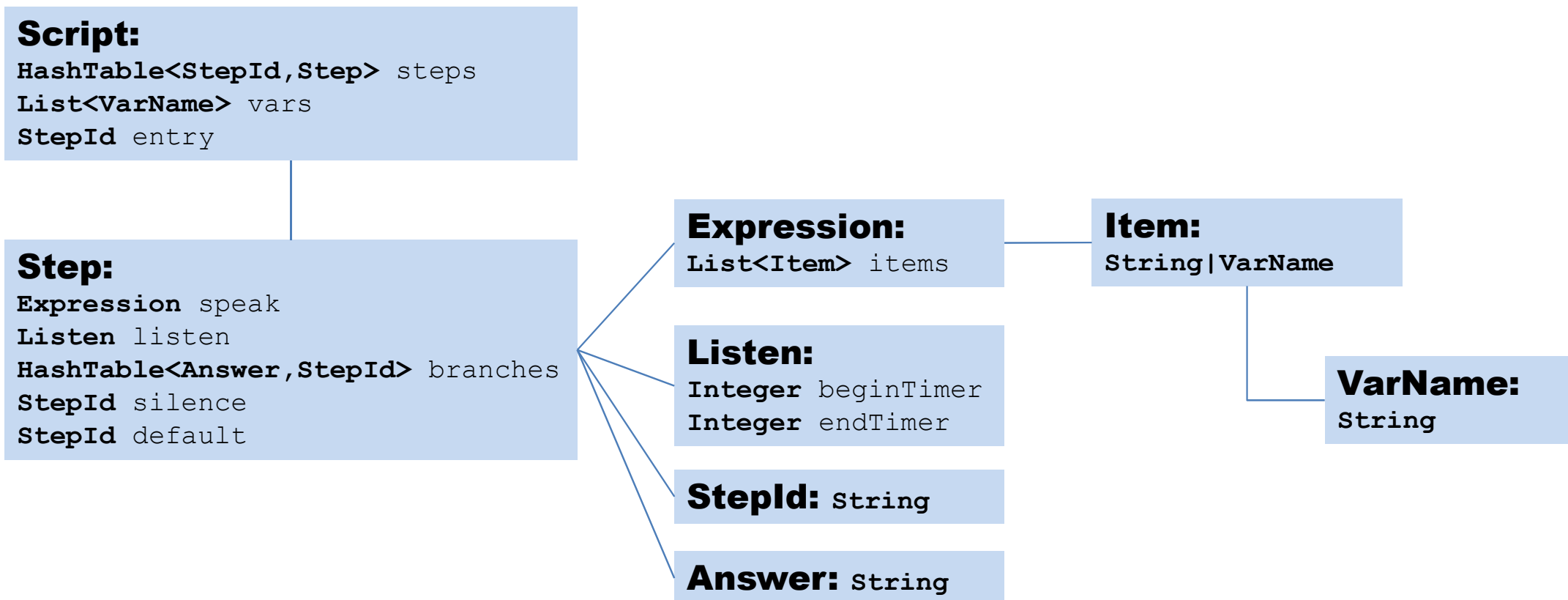
- 索引无规律
- 数据多，无序
- 数据经常变化

如何选择？

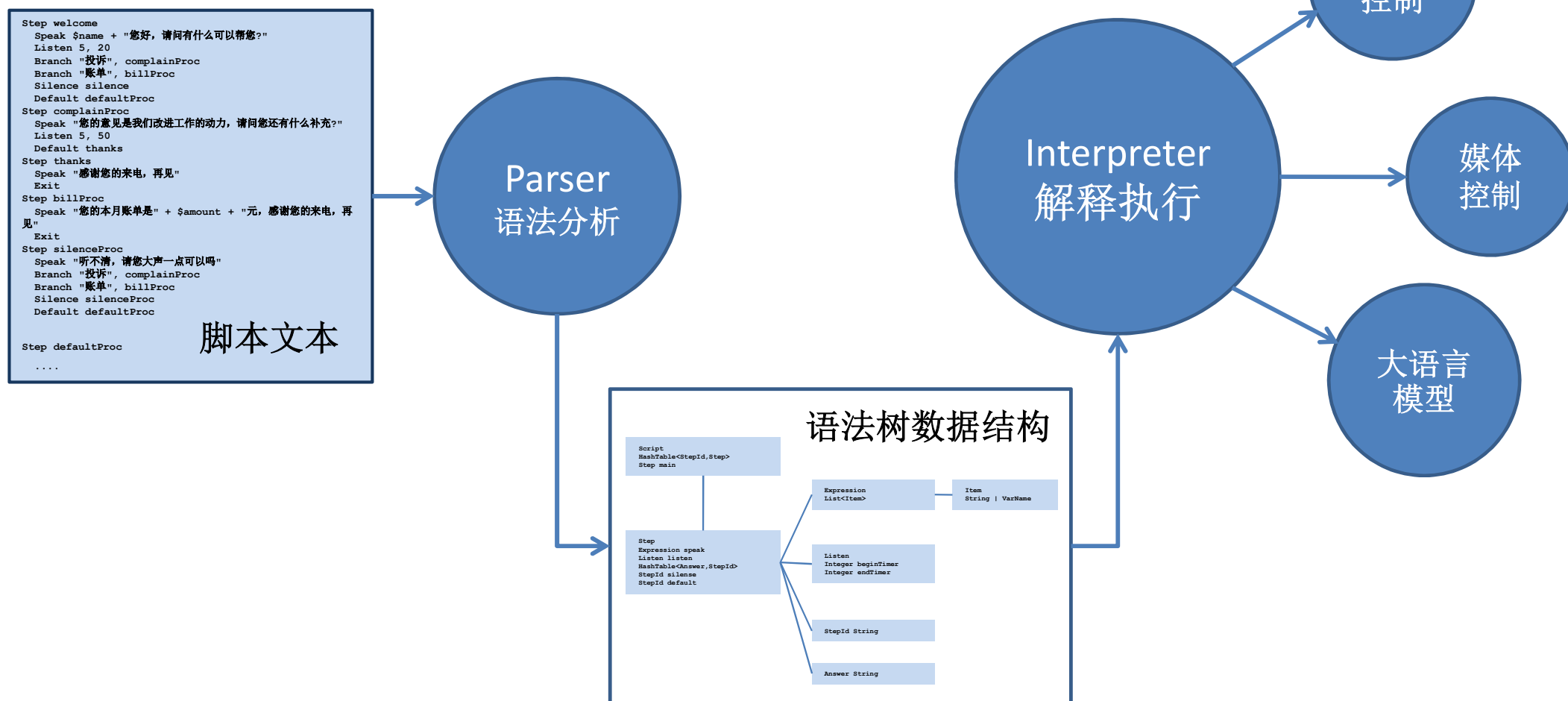
- 要处理的数据有多少？
- 数据量在程序运行过程中会有多大的变化？
- 数据索引以及内容的分布有什么特点？
- 数据的更新的频繁程度？
- 数据的查询的频繁程度？
- 有现成的库可用吗？
- 程序的运行环境有什么限制？
- ...



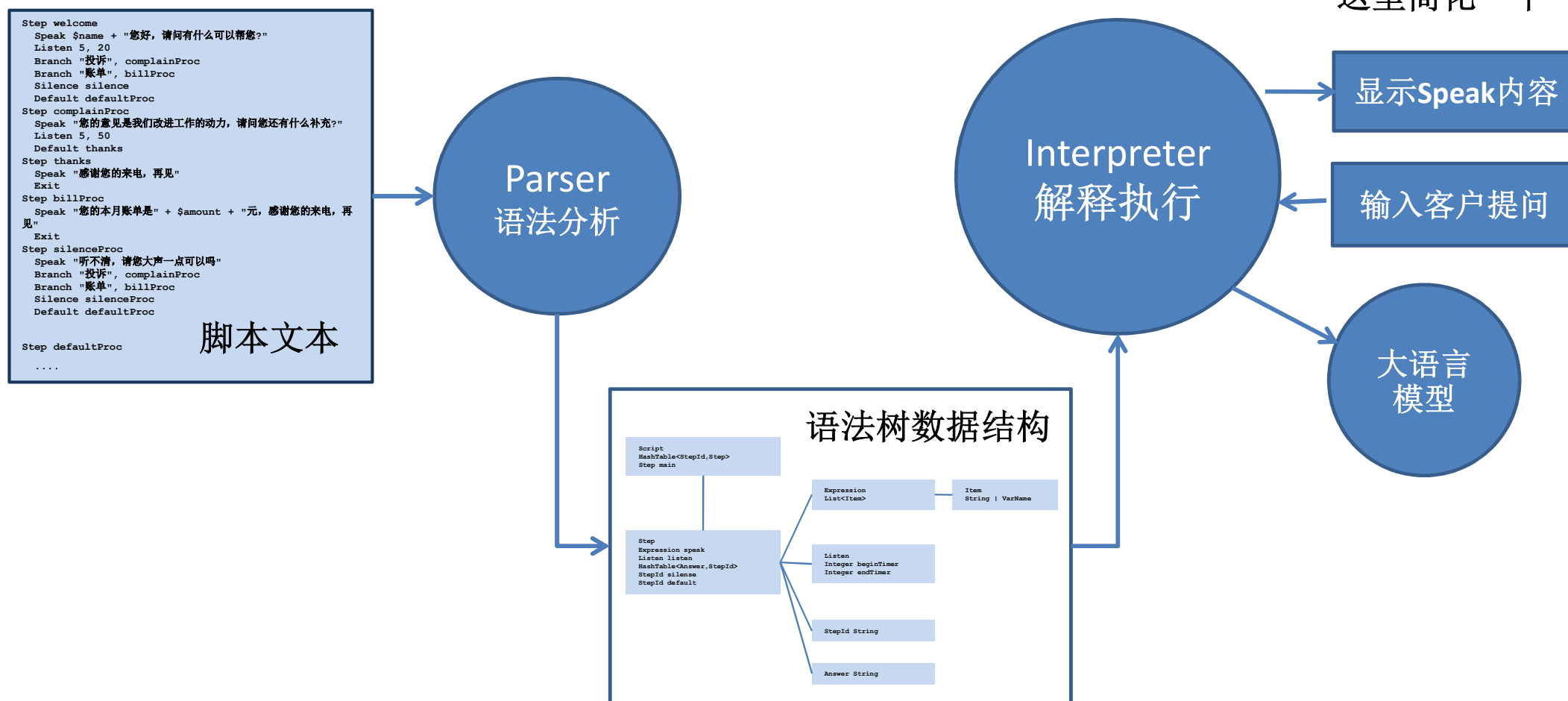
存储语法树的数据结构



程序总体结构



程序总体结构



Parser的实现

```
Step welcome
Speak $name + "您好, 请问有什么可以帮您?"
Listen 5, 20
Branch "投诉", complainProc
Branch "账单", billProc
Silence silence
Default defaultProc
Step complainProc
Speak "您的意见是我们改进工作的动力, 请问您还有什么补充?"
Listen 5, 50
Default thanks
Step thanks
Speak "感谢您的来电, 再见"
Exit
Step billProc
Speak "您的本月账单是" + $amount + "元, 感谢您的来电, 再见"
Exit
Step silenceProc
Speak "听不清, 请您大声一点可以吗?"
Branch "投诉", complainProc
Branch "账单", billProc
Silence silenceProc
Default defaultProc
Step defaultProc
.....
```

脚本文件

ParseFile

行

ParseLine

Tokens (词法元素)

标识符, 字符串, 整数等

根据每行的第一个Token调用不同的处理函数

StepProcess

SpeakProcess

BranchProcess

...

创建新的Step

更新当前Step

更新当前Step

语法树数据结构

```
Script
  HashTable<StepId, Step>
  Step main
  List<VarName>
```

```
Step
  Expression speak
  Listen listen
  HashTable<Answer, StepId>
  StepId silence
  StepId default
```

```
Expression
  List<Item>
```

```
Item
  String | VarName
```

```
Listen
  Integer beginTimer
  Integer endTimer
```

```
StepId String
```

```
Answer String
```

Parser的实现

ParseFile(fileName) :

打开文件

读取文件的每一行line:

`line.trim()` 删除行首空白

忽略空行

忽略'#'开头的注释行

ParseLine(line) 处理一行

关闭文件

ParseLine(line) :

读取一行中空白分割的每一个token:

遇到'#'开头的token则处理结束（忽略行尾注释）

获得标识符，字符串或者操作符几类token

将token加入到List中

ProcessTokens(token[])

Parser的实现

ProcessTokens (token[]) :

对List中的每一个token进行处理

根据token[0]分情况处理:

Step: ProcessStep (token[1])

Speak: ProcessSpeak (token+1)

Listen: ProcessListen (token[1], token[2])

Branch: ProcessBranch (token[1], token[2])

Silence: ProcessSilence (token[1])

Default: ProcessDefault (token[1])

Exit: ProcessExit ()

如果不是上述token则报错

ProcessStep (stepId) :

Script创建一个新的Step, 标识为stepId

设置当前Step为新创建的Step

如果这是第一个Step, 则设置当前Step为Script的entry

Parser的实现

ProcessSpeak(token[]):

token[]是一个表达式, 每个token可能是字符串, 变量或者 '+'
ProcessExpression(token[]) 得到Expression
将Speak以及对应的表达式存入当前的Step

ProcessExpression(token[]):

这么简单的表达式...
忽略掉加号, 其它token追加到Expression中的List<Item>
将变量名存入Script的List<VarName>中

ProcessListen(startTimer, stopTimer):

构造Listen(startTimer, stopTimer) 存入当前Step

ProcessBranch(answer, nextStepId):

将answer和nextStepId插入当前Step的HashTable

Parser的实现

ProcessSilence (nextStepId) :

将当前Step的silence变量设置成nextStepId中的值

ProcessDefault (nextStepId) :

将当前Step的default变量设置成nextStepId中的值

ProcessExit() :

将当前Step设置为终结Step

Interpreter的实现

执行环境:

- 变量表
- 当前Step
- ...

解释程序:

接通电话, 连接媒体服务器, 获取脚本语法树, 创建执行环境

当前Step置为entryStep

循环针对当前Step做:

执行Speak (语音合成, 语音播放)

如果本步骤是终结步骤, 则结束循环, 断开通话

执行Listen (录音, 语音识别, 自然语言意图分析)

获得下一个StepId:

如果用户沉默, 则获得Silence的StepId

根据用户意向查找HashTable, 获得StepId

如果查不到则获得Default的StepId

将当前Step置为刚才获得的StepId对应的Step

语法树数据结构

```
Script
  HashTable<StepId, Step>
  Step main
  List<VarName>
```

```
Step
  Expression speak
  Listen listen
  HashTable<Answer, StepId>
  StepId silence
  StepId default
```

```
Expression
  List<Item>
```

```
Item
  String | VarName
```

```
Listen
  Integer beginTimer
  Integer endTimer
```

```
StepId String
```

```
Answer String
```


Interpreter的实现（简化版本）

执行环境:

- 变量表
- 当前Step
- ...

解释程序（简化版）：

获取脚本语法树，创建执行环境

当前Step置为entryStep

循环针对当前Step做：

执行Speak（输出到标准输出）

如果本步骤是终结步骤，则结束循环，断开通话

执行Listen（直接从标准输入读入用户自然语言提问，调用进行意图分析）

获得下一个StepId：

如果用户沉默，则获得Silence的StepId

根据用户意向查找HashTable，获得StepId

如果查不到则获得Default的StepId

将当前Step置为刚才获得的StepId对应的Step

语法树数据结构

Script
HashTable<StepId, Step>
Step main
List<VarName>

Step
Expression speak
Listen listen
HashTable<Answer, StepId>
StepId silence
StepId default

Expression
List<Item>

Item
String | VarName

Listen
Integer beginTime
Integer endTime

StepId String

Answer String

Interpreter的执行环境

当两个用户同时和机器人对话，解释器需要两个线程同时运行，每个线程服务一个用户。思考一下，哪些东西还是一份？那些东西需要两份？

脚本文件：同一份（两个用户执行的是同一个脚本）

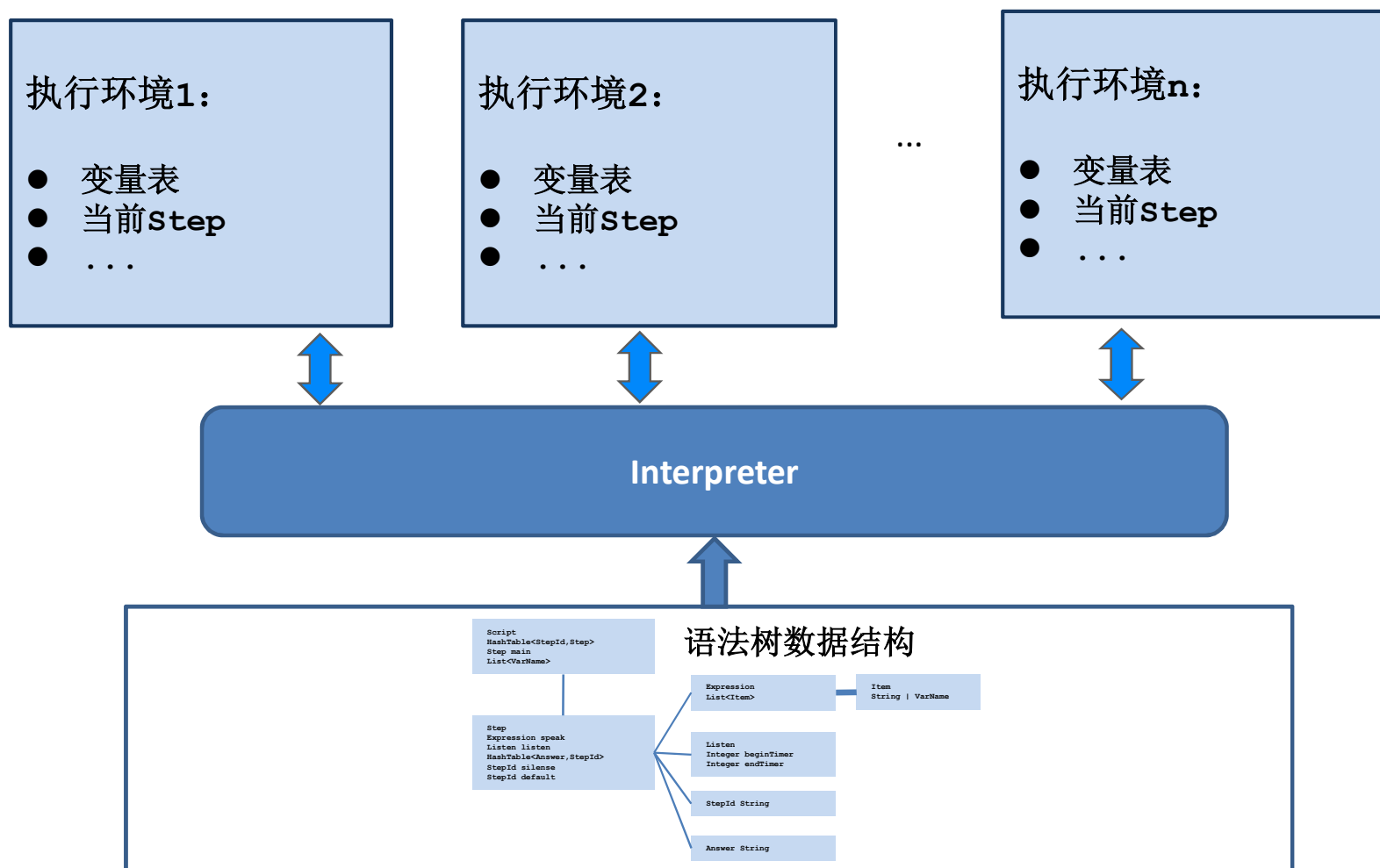
脚本语法树：同一份（可以调用一次Parser形成，多个线程公用）

但是，不同的用户肯定有不同的姓名，不同的账单，所以表示姓名和账单的变量表，肯定是两份。

两个用户有先有后，有快有慢，脚本执行的当前状态（例如当前step）肯定是两份。

简单来说，上述这些有“两份”的数据，也就是Interpreter的每次执行都需要一个新的实例的数据，我们称之为Interpreter的执行环境，需要单独的数据结构存放。

Interpreter的执行环境



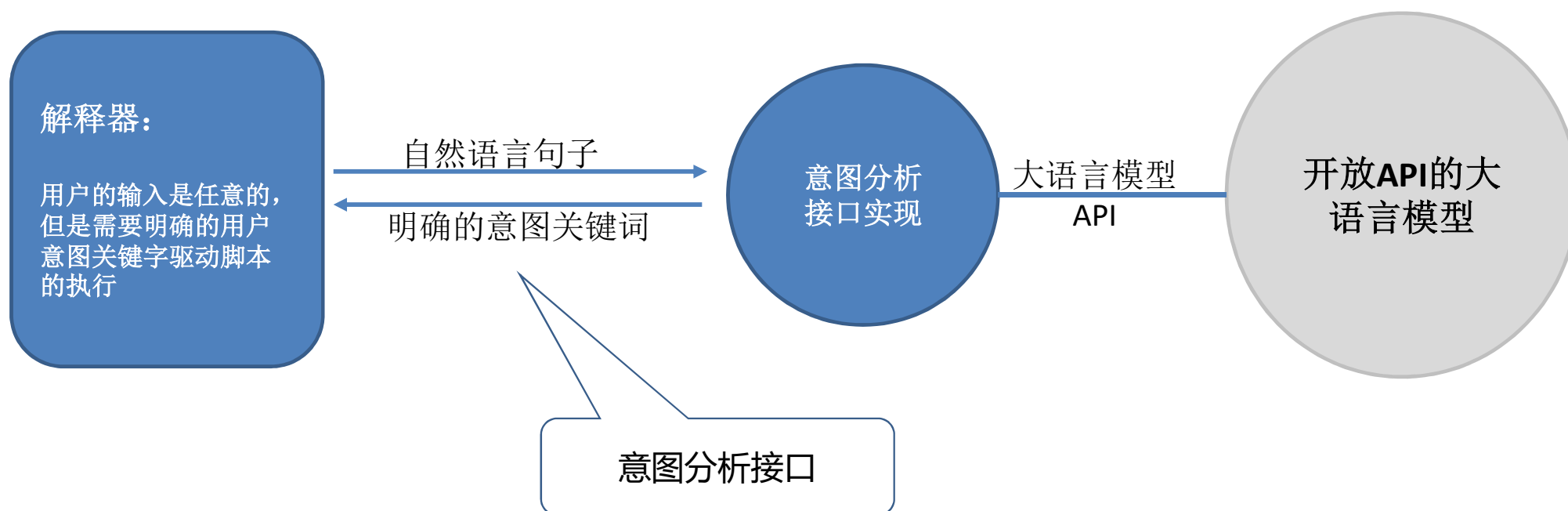
思考题

执行环境：

- 变量表 ?
- 当前Step

执行环境中的变量表应该是什么数据结构？如何构建？如何使用？

意图分析接口



意图分析接口

意图分析接口实现模块

